



System Design Tutorial

Phase 3 — Designing Real Systems, Step by Step

Six complete walkthroughs from requirements to trade-offs

Volume 3 of 4 | URL Shortener · News Feed · Chat · Video Streaming · Ride-Sharing · Rate Limiter

Table of Contents

The Walkthrough Method	6
Design 1 — A URL Shortener (like bit.ly)	7
Step 1 — Requirements	7
Functional	7
Non-functional	7
Step 2 — Estimation	7
Step 3 — Components	7
Step 4 — Deep Dive: generating the short key	7
Approach A — Hash the URL	8
Approach B — Encode a counter in Base62	8
Approach C — A Key Generation Service (KGS)	8
Step 5 — Trade-offs	8
301 vs 302 redirect — the signature trade-off	8
Design 2 — A Social Media News Feed (like Twitter/Instagram)	10
Step 1 — Requirements	10
Functional	10
Non-functional	10
Step 2 — Estimation	10
Step 3 — Components	10
Step 4 — Deep Dive: fan-out on write vs read	11
Fan-out on write (push model)	11
Fan-out on read (pull model)	11
The hybrid model (what real systems do)	11
Step 5 — Trade-offs	11
Design 3 — A Chat Application (like WhatsApp)	13
Step 1 — Requirements	13
Functional	13
Non-functional	13
Step 2 — Estimation	13

Step 3 — Components	13
Step 4 — Deep Dive: delivery, routing, and ordering	14
Finding the recipient	14
Delivery guarantees and acknowledgements	14
Ordering	14
Group chat	14
Step 5 — Trade-offs	14
Design 4 — A Video Streaming Platform (like YouTube/Netflix)	16
Step 1 — Requirements	16
Functional	16
Non-functional	16
Step 2 — Estimation	16
Step 3 — Components	16
Step 4 — Deep Dive: transcoding and adaptive bitrate streaming	17
Why transcode?	17
Adaptive Bitrate Streaming (ABR)	17
The role of the CDN	17
Step 5 — Trade-offs	17
Design 5 — A Ride-Sharing Service (like Uber)	19
Step 1 — Requirements	19
Functional	19
Non-functional	19
Step 2 — Estimation	19
Step 3 — Components	19
Step 4 — Deep Dive: finding nearby drivers (geospatial indexing)	20
Geohashing	20
Handling the write storm	20
Step 5 — Trade-offs	20
Design 6 — A Distributed Rate Limiter	22
Step 1 — Requirements	22
Functional	22
Non-functional	22

Step 2 — Where it lives	22
Step 3 — The core problem: distributed counting	22
Step 4 — Deep Dive: implementing it correctly	22
Atomic counting in Redis	22
Choosing the algorithm	23
Failure handling: fail-open vs fail-closed	23
Step 5 — Trade-offs	23

Phase 3 — Key Takeaways 24

The Walkthrough Method

Theory becomes skill only when you apply it. In this phase we design six real systems — the exact kind you will be asked to design in interviews — using one repeatable method.

Every design in this volume follows the same five steps, in the same order. Learning the *method* matters more than memorising any single design, because the method is what you reuse when you are handed a brand-new problem in an interview room.

- **1. Requirements** — pin down what the system must do (functional) and how well it must do it (non-functional: scale, latency, availability). Always clarify scope first.
- **2. Estimation** — back-of-the-envelope maths for traffic, storage, and bandwidth. This reveals where the hard problems are and justifies your choices.
- **3. Components & high-level design** — sketch the boxes and arrows: the major services and how data flows between them.
- **4. Deep dive** — zoom into the one or two genuinely hard parts and solve them in detail. This is where interviews are won.
- **5. Trade-offs** — state honestly what your design gives up, and what you'd change under different constraints.

A note on the numbers

- All estimates here are illustrative, rounded for easy mental maths — interviewers want sound reasoning, not precise figures.
- Handy constants: ~100K seconds in a day; 1 million = 10^6 ; 1 billion = 10^9 ; 1 character $\approx$ 1 byte.
- Always state assumptions out loud — 'assume 100M daily users' — so your maths is checkable.

Design 1 — A URL Shortener (like bit.ly)

The classic warm-up. Simple on the surface, but it teaches key generation, read-heavy caching, and the redirect trade-off — ideas that reappear everywhere.

Step 1 — Requirements

Functional

- Given a long URL, return a much shorter unique URL.
- When a user visits the short URL, redirect them to the original long URL.
- Optional: custom aliases, link expiration, basic click analytics.

Non-functional

- **Highly available** — if redirection is down, every link everywhere breaks.
- **Very low latency** redirects — users feel any delay.
- **Read-heavy** — links are created once but visited many times (assume ~100:1 reads to writes).
- Short links must be **unique** and effectively **non-guessable**.

Step 2 — Estimation

Assume **100 million new URLs per day** and a 100:1 read-to-write ratio.

- **Writes:** 100M / 100K seconds $\approx$ **1,000 writes/sec**.
- **Reads:** 100 $\times$ that $\approx$ **100,000 reads/sec** — the number that drives the design.
- **Storage:** keep links for 5 years $\rightarrow 100M \times 365 \times 5 \approx$ **180 billion URLs**. At ~500 bytes each (short key, long URL, metadata) that is roughly **90 TB**.

The takeaway: this is overwhelmingly a *read* problem. The write path can be modest; the read path must be fast and heavily cached.

Step 3 — Components

```

[Client] --> [Load Balancer] --> [App Servers] --> [Key-Value DB]
                                   |      (short_key -> long_url)
                                   v
                               [Redis Cache] (hot links)
                                   ^
redirect path: short_key -> cache hit -> 301/302 -> long URL
  
```

Figure D1.1 - A thin app tier over a key-value store, fronted by a cache.

The data model is trivially simple: a key-value mapping from `short_key` to `long_url` (plus owner, created_at, expiry). A key-value or wide-column store (DynamoDB, Cassandra) fits perfectly and scales horizontally. A cache (Redis) holds the most popular links so the bulk of the 100K reads/sec never touch the database.

Step 4 — Deep Dive: generating the short key

The heart of the problem: how do we turn each URL into a short, unique key? Three approaches:

Approach A — Hash the URL

Run the long URL through a hash (e.g. MD5) and take the first 7 characters. Simple, but two different URLs can collide on the same prefix, and the same URL always produces the same key. You must detect collisions and retry, which adds complexity.

Approach B — Encode a counter in Base62

Keep a global counter that increments for each new URL, and encode its value in **Base62** (a-z, A-Z, 0-9). This guarantees uniqueness with no collisions. With 7 characters you get $62^7 \approx 3.5$ trillion possible keys — plenty. The catch: a single global counter is a bottleneck and a single point of failure.

Approach C — A Key Generation Service (KGS)

Pre-generate a large supply of unique random keys offline and store them in a database of 'unused keys'. App servers grab keys in batches and hand them out. Generation is decoupled from the request path, uniqueness is guaranteed, and there is no live counter bottleneck. (Alternatively, give each server its own counter *range* — server 1 uses 1-1M, server 2 uses 1M-2M.)

BASE62 COUNTER counter++ -> encode62 "100000000" -> "6LAze" unique, but one counter is a bottleneck	KEY GENERATION SERVICE (KGS) [pool of pre-made unique keys] App servers pull batches v of keys to hand out no live bottleneck, guaranteed unique
--	---

Figure D1.2 - Base62 counter vs a pre-generated key pool.

Step 5 — Trade-offs

301 vs 302 redirect — the signature trade-off

	301 Permanent	302 Temporary
Browser caches it?	Yes — future clicks skip your server	No — every click hits your server
Server load	Lower	Higher
Analytics	You miss cached clicks	You see every click
Flexibility	Hard to change target later	Easy to change target

Use **301** to minimise load if analytics don't matter; use **302** if you need to count every click or change destinations. Many shorteners pick 302 precisely to keep analytics and control.

- **Caching:** cache-aside with LRU; the 80/20 rule means a small cache serves most traffic.
- **Database:** NoSQL key-value beats SQL here — the access pattern is a simple key lookup at huge scale, with no joins or transactions.
- **Custom aliases:** check uniqueness against existing keys before accepting.

URL shortener — remember this

- It is a **read-heavy** system: cache aggressively; the DB is a simple key-value lookup.
- Key generation: Base62 counter (simple, bottleneck) vs KGS (pre-generated pool, no bottleneck).
- 301 vs 302 is the trade-off interviewers probe: caching/load vs analytics/control.

Design 2 — A Social Media News Feed (like Twitter/Instagram)

The fan-out problem. This design hinges on one famous decision — do you build feeds when people post, or when people read? — and on how you handle celebrities.

Step 1 — Requirements

Functional

- Users can create posts (text, images, video).
- Users can follow other users.
- Each user sees a **feed**: recent posts from people they follow, newest (or most relevant) first.

Non-functional

- **Low latency** feed loads — the feed must appear almost instantly.
- **Highly available; eventual consistency** is acceptable (a post appearing a few seconds late is fine).
- **Massive read scale** — far more feed views than posts.

Step 2 — Estimation

Assume **500 million daily active users**, each opening their feed ~5 times a day, and **100 million posts/day**.

- **Feed reads**: $500M \times 5 = 2.5 \text{ billion/day} \approx \mathbf{25,000 \text{ reads/sec}}$ (peaks several times higher).
- **Posts (writes)**: $100M / 100K \approx \mathbf{1,000 \text{ posts/sec}}$.
- **The real cost is fan-out**: one post by a user with 1,000 followers may need delivering to 1,000 feeds. A celebrity with 100 million followers makes this explosive.

Step 3 — Components

```

[Post service] -> [Fan-out service] -> [Timeline cache (Redis)]
      |                (via queue)      ^
      v                | read          |
[Post store]      [Graph/Follow service] [Feed service] <- [Client]
media -> [Object storage] -> [CDN]

```

Figure D2.1 - Posting fans out into followers' timelines; reading serves a precomputed timeline.

- **Post service**: stores new posts; media goes to object storage and is served via CDN.
- **Graph service**: who follows whom.
- **Fan-out service**: on a new post, pushes it into followers' timelines (via a message queue so the poster isn't blocked).
- **Timeline cache**: each user's precomputed feed, held in Redis for instant reads.

- **Feed service:** reads the user's timeline and returns it.

Step 4 — Deep Dive: fan-out on write vs read

Fan-out on write (push model)

When a user posts, immediately insert the post ID into the timeline of every follower (precompute the feed). **Reads are then instant** — just fetch the ready-made list. Perfect for the read-heavy reality. The problem: a celebrity post triggers millions of writes (the **hot key / celebrity problem**), and inactive followers' feeds are computed for nothing.

Fan-out on read (pull model)

Don't precompute anything. When a user opens their feed, fetch recent posts from everyone they follow and merge them on the fly. **Writes are cheap** (store the post once), but **reads are expensive and slow** — gathering and sorting from hundreds of followees per feed load, at 25,000 reads/sec.

PUSH (fan-out on write)	PULL (fan-out on read)
post -> write into ALL followers' feeds	post -> store once
read -> instant (ready)	read -> gather from all followees, merge & sort at request time
bad for celebrities	slow reads, cheap writes

Figure D2.2 - Push precomputes feeds; pull builds them on demand.

The hybrid model (what real systems do)

Combine both. Use **push** for normal users (most posts, instant feeds), but for **celebrities** with huge follower counts, use **pull**: don't fan their posts out to millions; instead, when a user loads their feed, merge their precomputed timeline with a fresh pull of the few celebrities they follow. This caps the worst-case write explosion while keeping reads fast for everyone — essentially how Twitter handles timelines.

Step 5 — Trade-offs

- **Storage vs compute:** push spends storage (a post ID copied into every follower's feed) to save read compute; pull does the opposite. Hybrid balances them.
- **Freshness vs cost:** precomputed feeds may be slightly stale; acceptable given eventual consistency is fine here.
- **Ranking:** reverse-chronological is easy; a relevance-ranked feed (ML scored) is far more complex and usually computed at read time over a candidate set.
- **Pagination:** use cursor-based pagination ("posts before this ID/time"), not offset pages, so the feed stays stable as new posts arrive.

News feed — remember this

- The core decision is **fan-out on write (push)** vs **fan-out on read (pull)**.
- Push = instant reads, write explosion for celebrities. Pull = cheap writes, slow reads.
- Real systems use a **hybrid**: push for normal users, pull for celebrities, merged at read time.

Design 3 — A Chat Application (like WhatsApp)

Real-time, two-way, and unforgiving about lost messages. This design is all about persistent connections, finding the recipient, and delivery guarantees.

Step 1 — Requirements

Functional

- One-to-one messaging and group chats.
- Real-time delivery; messages arrive in order and are never lost.
- Delivery status (sent / delivered / read) and online / last-seen presence.
- Message history and offline delivery (messages wait for offline users).

Non-functional

- **Low latency** — messages should feel instant.
- **Reliable** — no message loss; correct ordering within a conversation.
- **Highly available** and able to hold **hundreds of millions of live connections**.

Step 2 — Estimation

Assume **500 million daily active users** sending **40 messages/day** each.

- **Messages:** $500M \times 40 = 20 \text{ billion/day} \approx \mathbf{200,000 \text{ messages/sec}}$.
- **Connections:** potentially hundreds of millions of *simultaneous* persistent connections — the defining infrastructure challenge.
- **Storage:** if each message is ~ 200 bytes, $20B/day \approx 4 \text{ TB/day}$ before media — media dominates and goes to object storage + CDN.

Step 3 — Components

Unlike request/response systems, chat needs a **persistent connection** so the server can push messages the instant they arrive. We use **WebSockets**. Clients hold an open connection to a fleet of **connection servers** (gateways).



Figure D3.1 - WebSocket gateways, a routing chat service, and a session registry.

- **Connection servers:** hold millions of open WebSocket connections; one job — ferry messages to and from connected clients.

- **Session registry** (Redis): maps each online user to the connection server they're attached to, so the system knows where to deliver.
- **Chat service**: receives a message, persists it, looks up the recipient's server, and forwards it.
- **Message store**: a wide-column store (Cassandra) keyed by conversation, time-ordered — built for huge write volume.
- **Notification service**: if the recipient is offline, send a push notification (APNs/FCM) and store the message for later delivery.

Step 4 — Deep Dive: delivery, routing, and ordering

Finding the recipient

When A messages B, the chat service consults the **session registry** to find which connection server holds B's live connection, then forwards the message there for instant push. If B has no live connection, B is offline: store the message and trigger a push notification so B's device wakes and fetches it.

Delivery guarantees and acknowledgements

Reliability comes from **acknowledgements**. The sender's message is stored before being confirmed 'sent'. When it reaches B's device, B's client sends an ack → status becomes 'delivered'. When B opens the chat, another ack → 'read'. If an ack doesn't arrive, the message is retried. Messages are kept until delivery is confirmed, guaranteeing none are lost.

```
A sends -> stored -> [SENT]
      -> pushed to B's device -> B acks -> [DELIVERED]
      -> B opens chat -> B acks -> [READ]
No ack? retry. Message persists until delivery is confirmed.
```

Figure D3.2 - Acknowledgements drive the sent/delivered/read state machine.

Ordering

Within a conversation, messages need a consistent order. Attach a **sequence number** or timestamp per conversation so the client can order and de-duplicate messages even if they arrive out of order over the network.

Group chat

A group message fans out to all members — for each member, route to their connection server if online, or queue + push if offline. For large groups this is a smaller-scale version of the feed fan-out problem.

Step 5 — Trade-offs

- **WebSockets vs polling**: WebSockets give true real-time and efficiency, at the cost of holding millions of stateful connections (hard to scale, needs careful connection management).

- **End-to-end encryption:** if messages are E2E encrypted (like WhatsApp), the server can route but not read them — great for privacy, but it complicates features like server-side search.
- **Store forever vs ephemeral:** some apps delete messages after delivery (privacy, cost); others keep full history (convenience). A product/cost decision.
- **Consistency:** per-conversation ordering matters; global ordering across all chats does not, which keeps the system scalable.

Chat app — remember this

- Use **WebSockets** for real-time push; clients connect to **connection servers**.
- A **session registry** (user → server) tells the system where to deliver each message.
- Reliability via **acks** and persistence; ordering via per-conversation sequence numbers.
- Offline users get a stored message plus a **push notification**.

Design 4 — A Video Streaming Platform (like YouTube/Netflix)

A storage and bandwidth monster. The two ideas that make it work are **transcoding** into many qualities and serving everything from a **CDN** close to users.

Step 1 — Requirements

Functional

- Upload videos; play videos smoothly on any device and connection speed.
- Search, recommendations, view counts, comments.

Non-functional

- **Low buffering / startup time** — playback must start fast and not stall.
- **Massive storage** and **enormous bandwidth**; global audience.
- **Highly available**; read-heavy (far more views than uploads).

Step 2 — Estimation

Assume **500 hours of video uploaded per minute** and billions of views per day.

- **Storage**: raw uploads are huge, and each is transcoded into ~5 resolutions plus multiple bitrates — multiplying storage several-fold. Petabytes accumulate quickly; object storage is the only economical home.
- **Bandwidth**: streaming video to billions is the dominant cost — which is exactly why a CDN is non-negotiable.
- **Read:write** ratio is extreme — a single popular video is viewed millions of times.

Step 3 — Components

```

UPLOAD PATH
[Uploader] -> [Upload svc] -> [Object storage (raw)] -> [Queue]
                                                    |
                                                    [Transcoding workers]
                                                    (make 240p..4K, chunks)
                                                    |
[Object storage (encoded)] -> [CDN]

PLAYBACK PATH
[Viewer] -> [CDN edge] (cached chunks) -> adaptive bitrate stream
metadata (title, etc.) from [Metadata DB]
```

Figure D4.1 - Upload triggers transcoding into many qualities; playback streams from the CDN.

- **Upload service**: accepts the raw file into object storage and enqueues a transcoding job.
- **Transcoding pipeline**: a queue feeds a fleet of workers that convert the video into multiple resolutions/bitrates and split each into small **chunks** (a few seconds each).
- **Object storage**: stores raw and encoded files cheaply and durably.

- **CDN**: caches video chunks at edge servers worldwide — the single most important component for playback.
- **Metadata DB**: titles, descriptions, view counts (SQL or NoSQL depending on scale).

Step 4 — Deep Dive: transcoding and adaptive bitrate streaming

Why transcode?

A single huge 4K file can't serve everyone: a phone on 3G needs a tiny, low-bitrate version; a TV on fibre wants 4K. So each upload is **transcoded** into a ladder of resolutions and bitrates (240p, 480p, 720p, 1080p, 4K), each chopped into short chunks. Transcoding is CPU-heavy and slow, so it runs asynchronously in the background — the uploader gets 'processing...' immediately while workers churn through the queue.

Adaptive Bitrate Streaming (ABR)

The magic behind smooth playback. The video is delivered as a sequence of small chunks available at every quality level (via **HLS** or **DASH**). The player measures current bandwidth and **switches quality chunk-by-chunk**: bandwidth drops → fetch lower-quality chunks to avoid stalling; bandwidth recovers → step back up. That's why a stream blurs briefly then sharpens instead of freezing.

```
ABR: video split into chunks at multiple qualities
1080p: [c1][c2][c3][c4]...
720p:  [c1][c2][c3][c4]...
480p:  [c1][c2][c3][c4]...
Player picks per chunk based on live bandwidth:
[1080p][1080p][480p (slow!)] [720p][1080p] -> smooth, no freeze
```

Figure D4.2 - The player adapts quality chunk-by-chunk to the available bandwidth.

The role of the CDN

Popular chunks are cached at edge servers near viewers, so the vast majority of bytes never travel from the origin. Netflix takes this furthest with **Open Connect**: it places its own caching appliances *inside ISPs*, so a stream often comes from a box in your own city — minimal latency, minimal backbone traffic.

Step 5 — Trade-offs

- **Storage vs experience**: storing many resolutions multiplies cost but is essential for smooth, device-appropriate playback.
- **Pre-transcode vs on-demand**: transcoding everything up front costs storage but guarantees instant playback at any quality; transcoding rarely-watched videos on demand saves storage but adds first-view delay. Hybrid: pre-transcode common qualities, generate rare ones lazily.
- **Build vs buy CDN**: use a commercial CDN to start; at Netflix/YouTube scale a custom CDN pays off massively.

- **View counts:** exact real-time counts are expensive at billions of views; approximate, eventually-consistent counters are the norm.

Video streaming — remember this

- **Transcode** each upload asynchronously into a ladder of resolutions/bitrates, split into chunks.
- **Adaptive bitrate streaming (HLS/DASH)** switches quality per chunk to prevent buffering.
- The **CDN** is the heart of playback — it serves cached chunks close to users.
- Object storage holds the (huge) files; metadata lives in a separate database.

Design 5 — A Ride-Sharing Service (like Uber)

Real-time geography at scale. The signature challenge is finding nearby drivers fast while millions of cars stream their location every few seconds.

Step 1 — Requirements

Functional

- Riders request a ride; the system matches them with a nearby available driver.
- Real-time location tracking of drivers and the trip.
- Trip lifecycle (request → matched → en route → complete), pricing (incl. surge), payment.

Non-functional

- **Low-latency matching** — riders won't wait long for a car.
- **High write volume** — every active driver sends frequent GPS updates.
- **Highly available**; correct trip state is important (consistency for the trip itself).

Step 2 — Estimation

Assume **5 million active drivers**, each sending a location update **every 4 seconds**.

- **Location updates**: $5M / 4 \approx 1.25 \text{ million writes/sec}$ — a torrent of small updates. This volume dictates the architecture.
- **Matching reads**: each ride request triggers a 'find drivers near me' geo-query.
- These updates are short-lived — we mostly care about *current* position, which points us to an in-memory geo-index rather than a disk database for the hot path.

Step 3 — Components

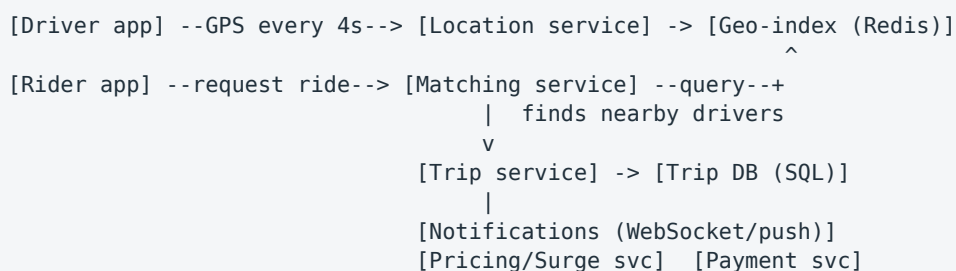


Figure D5.1 - A high-write location pipeline feeds a geo-index that the matcher queries.

- **Location service**: ingests the flood of GPS updates and writes current positions into an in-memory **geo-index**.
- **Matching service**: on a ride request, queries the geo-index for nearby available drivers and selects one.
- **Trip service**: manages the trip's state machine; trip data is important, so it lives in a consistent (SQL) store.
- **Pricing/surge service**: computes fares and demand-based surge multipliers.

- **Notification service:** pushes real-time updates (driver location, ETA) to apps over WebSockets.

Step 4 — Deep Dive: finding nearby drivers (geospatial indexing)

The core problem: given a rider's location, quickly find available drivers within, say, 2 km — out of millions, updating constantly. Scanning every driver is impossible at 1.25M updates/sec. We need a **geospatial index** that makes 'who is near here?' fast.

Geohashing

Geohashing divides the world into a grid of cells and encodes each cell as a short string; nearby locations share a common prefix. To find nearby drivers, you compute the rider's geohash and look up drivers in the same and adjacent cells — turning an expensive distance search into a fast prefix/key lookup. Redis offers built-in geospatial commands that do exactly this.

```

Divide the map into cells; each cell has an ID (geohash).
+-----+-----+-----+      Rider is in cell "gcpv".
| gcpu | gcpv | gcpv |      Look up drivers in "gcpv" and its
+-----+---R---+-----+      8 neighbours -> a small candidate set,
| gcps | gcpt | gcpw |      instead of scanning all 5M drivers.
+-----+-----+-----+

```

Figure D5.2 - Geohashing turns 'find nearby' into a cheap grid-cell lookup.

Alternatives include **QuadTrees** (recursively subdivide busy areas into smaller cells, balancing density) and Google's **S2** library. All share the same goal: avoid comparing the rider against every driver, and instead narrow to a small local set.

Handling the write storm

1.25M updates/sec would crush a traditional database, so current locations live **in memory** (Redis), where the data is transient and frequently overwritten. We don't durably store every single GPS ping on the hot path — we keep the latest position for matching, and stream the raw history to a separate analytics pipeline (e.g. Kafka) for later use.

Step 5 — Trade-offs

- **Update frequency vs load:** more frequent GPS updates mean fresher positions but more write load. 4 seconds is a balance; you can adapt it (less frequent when idle).
- **In-memory vs durable:** the geo-index trades durability for speed — fine, because a lost position is replaced within seconds by the next update.
- **Matching strategy:** nearest driver is simplest, but real systems optimise globally (ETAs, driver acceptance, fairness) — a harder optimisation problem.
- **Consistency split:** location data is best-effort (AP); trip and payment state must be correct (CP) — a great example of using different guarantees for different data.

Ride-sharing — remember this

- The hard part is **real-time geospatial search**: use **geohashing** / QuadTrees to find nearby drivers fast.
- A huge **write volume** of GPS updates lives **in memory** (Redis geo-index), not on disk.
- Split consistency: locations are best-effort (AP); trip and payment data are strongly consistent (CP).

Design 6 — A Distributed Rate Limiter

*A focused, deep design. We met the algorithms in Phase 2; here the challenge is making rate limiting work **accurately across many servers** without becoming a bottleneck itself.*

Step 1 — Requirements

Functional

- Limit each client to N requests per time window (e.g. 100 requests/minute).
- Reject excess requests with HTTP 429 Too Many Requests.
- Support different rules per client tier and per endpoint.

Non-functional

- **Low latency** — the limiter sits in front of every request, so it must add almost no delay.
- **Accurate** across a distributed fleet of servers.
- **Highly available** — and it must fail gracefully if its own datastore has trouble.

Step 2 — Where it lives

Rate limiting belongs at the **edge** — in the API gateway or a reverse proxy — so rejected traffic is stopped before it consumes application resources. The gateway runs on many instances for scale, which immediately creates the central problem.

Step 3 — The core problem: distributed counting

If each gateway instance counted requests in its own local memory, a client allowed 100/min hitting 10 instances could send 1,000/min — each instance sees only its slice. The counters must be **shared**. The standard solution is a fast central store, almost always **Redis**, holding one counter per client.

```
[Client] -> [Gateway 1] --+
[Client] -> [Gateway 2] --+--> [ Redis: per-client counters ]
[Client] -> [Gateway 3] --+          INCR + EXPIRE (atomic)
                                   count > limit ? -> 429
```

Shared counters in Redis so all gateways enforce the SAME limit.

Figure D6.1 - All gateway instances share counters in Redis for one global limit.

Step 4 — Deep Dive: implementing it correctly

Atomic counting in Redis

For a fixed-window limit, each request does INCR key (atomically increment this client's counter) and, on the first request of a window, sets an EXPIRE so the counter resets automatically. If the value exceeds the limit, return 429. Atomic operations (or a small Lua script run inside Redis) prevent **race conditions** where two gateways read-then-write the same counter and both think they're under the limit.

Choosing the algorithm

The algorithm choice (from Phase 2) maps onto this design: **token bucket** (store tokens + last refill time per client; allows bursts) is the popular default; **sliding window** avoids the fixed-window boundary burst where a client sends the full limit at the end of one window and the start of the next. Both can be implemented as a short atomic Redis script.

```
FIXED-WINDOW BOUNDARY PROBLEM (why sliding window exists)
limit = 100/min
:59 -> send 100    (allowed, window A)
:00 -> send 100    (allowed, window B)
=> 200 requests in ~2 seconds across the seam!
Sliding window counts the last 60s continuously -> no burst.
```

Figure D6.2 - Fixed windows allow a double-rate burst at the boundary; sliding windows fix it.

Failure handling: fail-open vs fail-closed

What if Redis is briefly unavailable? **Fail-open** (allow requests through) keeps your service usable but temporarily unprotected; **fail-closed** (reject) stays protected but may block legitimate users. Most systems **fail open** for rate limiting — a brief lapse in limiting is less harmful than an outage — but this is a deliberate, stated choice.

Step 5 — Trade-offs

- **Accuracy vs latency:** a central Redis counter is accurate but adds a network hop per request. An alternative is local approximate counting that periodically syncs — faster, slightly less exact.
- **Algorithm:** token bucket allows bursts (often desirable); sliding window is smoother and more precise; fixed window is simplest but bursty at boundaries.
- **Fail-open vs fail-closed:** availability vs protection — pick per the cost of each failure mode.
- **Granularity:** limit per user, per IP, per API key, or per endpoint — or combinations.

Rate limiter — remember this

- Enforce at the **edge** (gateway); the hard part is **shared, accurate counters** across many instances.
- Use **Redis** with **atomic** operations (INCR/EXPIRE or a Lua script) to avoid race conditions.
- Know token bucket vs sliding vs fixed window, and the **fail-open vs fail-closed** decision.

Phase 3 — Key Takeaways

Six designs, one method. If you can recall the signature challenge of each system below, you can rebuild the whole design from it.

The method (reuse this for ANY design)

- **Requirements** (functional + non-functional) → **Estimation** (traffic, storage, bandwidth) → **Components** (boxes & arrows) → **Deep dive** (the hard part) → **Trade-offs**.
- Estimation reveals where the difficulty is; the deep dive is where interviews are won.

Each design's signature challenge

- **URL Shortener**: read-heavy caching + key generation (Base62 vs KGS) + 301/302 trade-off.
- **News Feed**: fan-out on write vs read; hybrid (push for normal users, pull for celebrities).
- **Chat App**: persistent WebSocket connections + session registry + acks for reliable delivery.
- **Video Streaming**: async transcoding into a quality ladder + adaptive bitrate + CDN at the core.
- **Ride-Sharing**: real-time geospatial search (geohashing) + in-memory geo-index for the write storm.
- **Rate Limiter**: shared atomic counters in Redis at the edge; algorithm + fail-open/closed choice.

Patterns that recur across all six

- **Cache the hot path**, push heavy/slow work to **asynchronous queues**, and serve static/media via **object storage + CDN**.
- **Split consistency by data type**: best-effort (AP) for ephemeral data, strong (CP) for money and critical state.
- Identify the **one hard problem** early and spend your time there — that is the design.

What comes next

- **Phase 4** is interview preparation: 40 questions across three difficulty levels, a universal framework for approaching any prompt, the common mistakes to avoid, and how to communicate your thinking clearly under pressure.